

Podman in openSUSE Tumbleweed / Leap

Daemonless Containers — Why openSUSE Ships Podman as Default

Didiet Agus Pambudiono

Site Reliability Engineer / DevOps Engineer

openSUSE Asia Summit 2026 · 30 min incl. Q&A

About Me

Didiet Agus Pambudiono — SRE / DevOps Engineer

- **Work:** do some magic things as a Vibe Coder Full Stack and Snack DevSecOps Engineer @ Friendsure.tech
- Daily driver: **openSUSE Tumbleweed**
- Runs homelab infrastructure-as-code: Proxmox + Terraform + Ansible
- Kubernetes (k8s + K3s HA), GitLab CI, Prometheus monitoring
- Container workflows: Podman, Buildah, Docker — in prod and at home

 **Why this talk?** People keep asking: “Docker works — why did openSUSE choose Podman?”

Agenda — 30 minutes

Section	Time
Why the default matters	3 min
Architecture: daemonless vs client–server	4 min
Rootless containers out of the box	4 min
Getting started on Tumbleweed & Leap	4 min
Pods & systemd integration	4 min
Buildah, Compose, networking, journald	4 min
Q&A	7 min

Why the Default Matters

Your distro's default container engine shapes your muscle memory:

- What's preinstalled, documented, and tested together
- What `zypper install` gives you with zero extra repos
- What integrates cleanly with **systemd**, **journald**, and **firewalld**

openSUSE ships **Podman** as the default — daemonless, rootless-first, OCI-compliant.

This talk: what that means in practice on **Tumbleweed** (rolling) and **Leap** (stable).

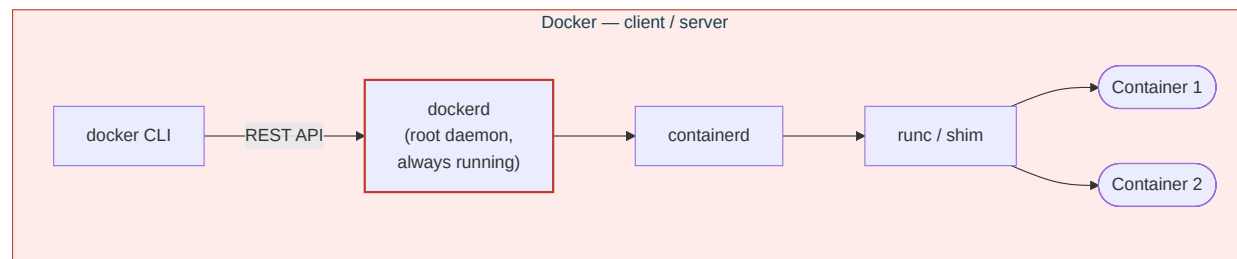
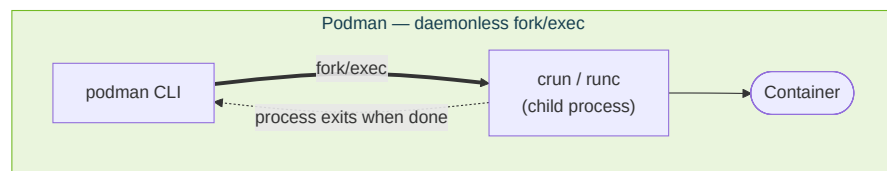
What Is Podman?

An open-source container engine — **Red Hat–originated, community-driven**

- Manages the full OCI lifecycle: **run · build · push · pull**
- **Alias-compatible with Docker CLI** — same flags, same workflow
- **No daemon required** — plain processes, managed by your init system
- Ships as `containers-*` ecosystem: Podman + Buildah + Skopeo

```
$ sudo zypper install podman          # that's really it
$ alias docker=podman                # optional – or zypper install podman-docker
```

Architecture: Docker vs Podman



- Docker: one long-lived **root daemon** owns every container
- Podman: each command spawns a short-lived **runtime child process**

Why Daemonless Matters

Concern	With a daemon (Docker)	Daemonless (Podman)
Attack surface	Persistent root daemon	Nothing resident between commands
Daemon crash	All containers die	Processes unaffected
systemd integration	Needs workarounds	Native — containers are plain processes
Resource idle cost	Daemon always in memory	Zero footprint between commands

No daemon = no single point of failure, no standing privileged process.

Rootless by Default

Containers run as **your UID**, inside a **user namespace**

```
host:  didiet (1000) → namespace: root (0) → container sees itself as root
```

- No privileged escalation just to pull and run an image
- Files created in volumes are owned by **you**, not root
- Compromised container escapes into... an unprivileged user. That's it.
- Works identically on **Tumbleweed** and **Leap** — nothing to enable

```
$ podman info --format '{{.Host.Security.Rootless}}'  
true
```

What openSUSE Preconfigures for You

Rootless needs subordinate UIDs/GIDs — **already done**:

- `/etc/subuid` & `/etc/subgid` ranges auto-assigned at user creation
- `shadow-utils` + `uidmap` installed as dependencies
- `fuse-overlayfs` fallback if kernel overlay isn't available rootless
- Default registries in `/etc/containers/registries.conf`

```
# /etc/containers/registries.conf (excerpt)
unqualified-search-registries = ["registry.opensuse.org", "docker.io"]
```

registry.opensuse.org — official openSUSE base images, first in line.

Getting Started on Tumbleweed & Leap

```
# Tumbleweed – always current Podman (5.x)
sudo zypper install podman podman-compose buildah skopeo

# Leap – same packages, stable cadence
sudo zypper install podman

# Verify
podman version && podman info
```

First container:

```
podman run --rm registry.opensuse.org/opensuse/tumbleweed cat /etc/os-release
podman run -dt -p 8080:80 --name web nginx
# use 127.0.0.1 – localhost may resolve to IPv6 ::1 (rootless)
curl http://127.0.0.1:8080
```

Gotcha: registry.opensuse.org Namespaces

openSUSE images live under the `opensuse/` namespace — the bare path does not exist:

```
podman pull registry.opensuse.org/nginx          # ✗ name unknown
podman pull registry.opensuse.org/opensuse/nginx # ✓ correct
podman pull nginx                                # ✓ short name
```

Fix it once in `~/.config/containers/registries.conf`:

```
[aliases]
"nginx" = "registry.opensuse.org/opensuse/nginx"

[[registry]]
prefix = "registry.opensuse.org/nginx"
location = "registry.opensuse.org/opensuse/nginx"
```



Live Demo 1

Run, inspect, stop — all as an unprivileged user

```
podman ps · podman logs web · podman stop web
```

Fallback: terminal screencast

Pods: Groups That Map to Kubernetes

A **pod** = containers sharing network, IPC, and UTS namespaces

```
podman pod create --name stack -p 8080:80
podman run -d --pod stack --name app my-app
podman run -d --pod stack --name sidecar log-exporter
```

Kubernetes-ready round-trip:

```
podman kube generate stack -f stack.yaml    # export pod → K8s YAML
podman kube play stack.yaml                 # run K8s YAML locally
```

Develop locally with Podman, hand the **same YAML** to your cluster.

systemd Integration: Containers as Services

The modern way — **Quadlet** (Podman ≥ 4.4):

```
# ~/.config/containers/systemd/web.container
[Container]
Image=registry.opensuse.org/opensuse/nginx
PublishPort=8080:80
Volume=/srv/www:/usr/share/nginx/html:ro
AutoUpdate=registry

[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload && systemctl --user start web
```

Real units, real logs, real dependencies — `After=`, `Wants=`, restart policies.

Legacy: `podman generate systemd` still exists but is **deprecated** — prefer Quadlet.



Live Demo 2

Reboot-proof container service

```
loginctl enable-linger  
systemctl --user status web  
sudo reboot    # ...and it comes back
```

This is the moment people switch

Building Images: Buildah Under the Hood

`podman build` **is** Buildah — but Buildah also stands alone:

```
# Containerfile
FROM registry.opensuse.org/opensuse/tumbleweed
RUN zypper --non-interactive install python3 && zypper clean -a
COPY app.py /app/
CMD ["python3", "/app/app.py"]
```

```
podman build -t localhost/myapp .
buildah bud -t localhost/myapp .      # equivalent, scriptable
buildah from scratch                  # minimal image assembly, no Dockerfile
```

Same storage, same layers, no daemon building in the background.

Compose Workflows: podman-compose

Your existing `docker-compose.yml` mostly **just works**:

```
services:
  web:
    image: registry.opensuse.org/opensuse/nginx
    ports: ["8080:80"]
  db:
    image: docker.io/library/postgres:16
    environment: [POSTGRES_PASSWORD=example]
```

```
sudo zypper install podman-compose
podman-compose up -d
```

- Rootless, daemonless — each service becomes its own process/pod
- Full Docker Compose parity isn't the goal; common stacks are covered

Networking & Logging: openSUSE Defaults

netavark — Podman's native network stack (replaced CNI in v4)

```
podman network create lab
podman run -d --network lab --name db postgres
```

aardvark-dns — embedded DNS: containers resolve each other **by name**

journald — container stdout lands in the system journal:

```
journalctl --user -u web          # Quadlet-managed
podman logs -f web                # classic tailing
```

Everything queryable with tools sysadmins already know.

Podman vs Docker: Choosing

	Podman	Docker
Daemon	✗ none	✓ required
Rootless-first	✓ default	opt-in, more setup
License	Apache-2.0	Apache-2.0 (Engine)
Compose	podman-compose	full Compose
systemd services	✓ Quadlet, native	workarounds
CLI compatibility	drop-in	original

Honest take: either engine runs your OCI images. On openSUSE, Podman is the *path of least resistance* — preconfigured, integrated, supported.

Key Takeaways

1. **Daemonless ≠ gimmick** — fewer privileges, no daemon SPOF, native systemd
2. **Rootless out of the box** — openSUSE does the namespace setup for you
3. **Quadlet** turns containers into first-class boot-persistent services
4. **Same OCI images, same CLI habits** — switching costs near zero
5. Start today:

```
sudo zypper install podman && podman run --rm -it opensuse/tumbleweed bash
```

Resources

- Docs — <https://docs.podman.io>
- openSUSE containers portal — <https://registry.opensuse.org>
- Quadlet guide — `man quadlet` · `/usr/share/doc/packages/podman`
- These slides + demo files — <https://github.com/didietap/osas26-podman>

Thank you! Questions?

Get in Touch

- **GitHub** — <https://github.com/didietap/osas26-podman>
- **LinkedIn** — <https://www.linkedin.com/in/didiet-a-pambudiono>
- **Email** — pambudiono.os@gmail.com
- **Telegram** — [@didiet_ap](https://t.me/didiet_ap)

Note: please introduce yourself the first time you contact me.

 **Terima kasih!**

Q&A — 7 minutes

Didiet Agus Pambudiono · SRE / DevOps Engineer

Slides will be shared — come find me during the break too